

Applied Machine Learning and Deep Learning (MBA ZG582)

Deep Learning Phase

Entity-Embedding Neural Network for Bike-Buyer Propensity

Adventure Works Cycles / Microsoft AdventureWorks customer data

Framework	PyTorch 2.7.1+cu118 (device: cuda)
Architecture	Entity embeddings -> BN -> 128 -> 64 -> 1
Trainable parameters	12,406
Embedded features	9 categorical
Continuous features	6
Data split	11154 train / 1969 validation / 3281 test
Training	60 epochs run, best at epoch 45 (early stopping)
Neural network ROC-AUC	0.8548
Benchmark model	HistGradientBoosting (tuned) at 0.8606

1. Objective

The benchmark for this project is a tuned histogram gradient-boosting classifier, which reaches ROC-AUC 0.8606 on a held-out 20 per cent of customers. This report asks two questions of that result.

- Can a neural network with learned entity embeddings improve on the boosted trees, using the same data and the same split?
- Independently of the score, does the embedding representation reveal structure in the categorical variables that a tree ensemble cannot express?

The second question matters even if the answer to the first is no. A tree ensemble can tell you that occupation is important; it cannot tell you which occupations behave alike. An embedding can.

1.1 Why Entity Embeddings

One-hot encoding places every level of a categorical variable at an equal distance from every other. Under that encoding the model has no way to represent the idea that two occupations attract similar customers - each level is simply a separate, unrelated indicator column. An embedding layer instead assigns each level a short dense vector, learned by gradient descent alongside the rest of the network, so that levels which behave similarly for the prediction task end up close together in the embedding space.

Two practical consequences follow. The representation is more compact than one-hot for high-cardinality fields, and - more useful here - the learned vectors can be extracted and plotted, so the structure the network discovered is open to inspection rather than taken on trust.

2. Architecture and Training

Nine categorical fields are embedded and six continuous fields are passed through batch normalisation, then concatenated and fed to a two-layer feed-forward network. Embedding width follows the standard heuristic $\min(50, (\text{cardinality} + 1) / 2)$, which gives dimensions of 1 to 3 for the low-cardinality fields in this dataset.

Component	Detail
Embedding layers	9 categorical fields, dims 1-3, concatenated (27 dims total with continuous)
Continuous path	6 features, BatchNorm1d
Hidden layer 1	Linear 128 -> BatchNorm -> ReLU -> Dropout 0.35
Hidden layer 2	Linear 64 -> BatchNorm -> ReLU -> Dropout 0.20
Output	Linear 1, BCEWithLogitsLoss
Class imbalance	pos_weight in the loss - the network equivalent of class_weight='balanced'
Optimiser	AdamW, lr 2e-3, weight decay 1e-4, ReduceLROnPlateau on validation AUC
Early stopping	Patience 15 epochs on validation ROC-AUC
Parameters	12,406 trainable

The count fields - children at home, total children, cars owned - are embedded rather than passed through as integers. This lets the network learn its own spacing between, say, two children and three, instead of being forced into the assumption that the effect is linear in the count.

2.1 Guarding Against Divergence from the Benchmark

For the comparison to be meaningful, the network must see exactly the data the boosted trees saw. The preparation code mirrors the benchmark pipeline step for step - the same deduplication rule, the same removal of identifier and personally identifying columns, the same derived features, the same random seed and the same stratified 80/20 split - and asserts that the resulting row count matches 16,404, so any silent drift fails loudly rather than quietly biasing the result. A validation slice is carved from the training portion only, so the test set is never seen during training or model selection.

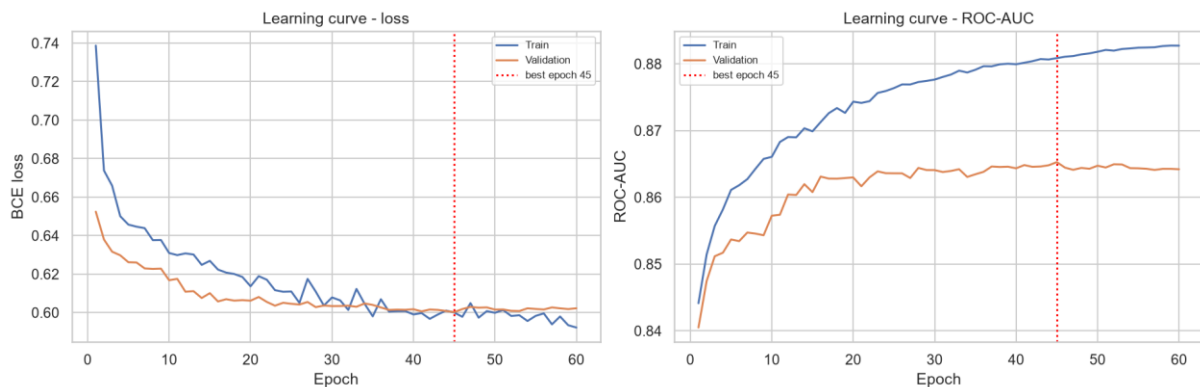


Figure 1: Learning curves. Training and validation track closely, and early stopping selects epoch 45.

The curves show no meaningful over-fitting: validation loss flattens rather than turning upward, and the gap to training loss stays narrow throughout. The regularisation - dropout, weight decay and batch normalisation - is doing its job on a dataset of this size.

3. Results

Model	Accuracy	BalancedAccuracy	Precision	Recall	F1	ROC_AUC	PR_AUC	Delta_ROC_AUC
HistGradientBoosting (tuned) (benchmark)	0.7998	0.7527	0.7406	0.6123	0.6703	0.8606	0.7791	0.0
Entity-Embedding Neural Network	0.7729	0.7685	0.6329	0.7553	0.6887	0.8548	0.7671	-0.0058

Table 1: Head-to-head on the identical hold-out split.

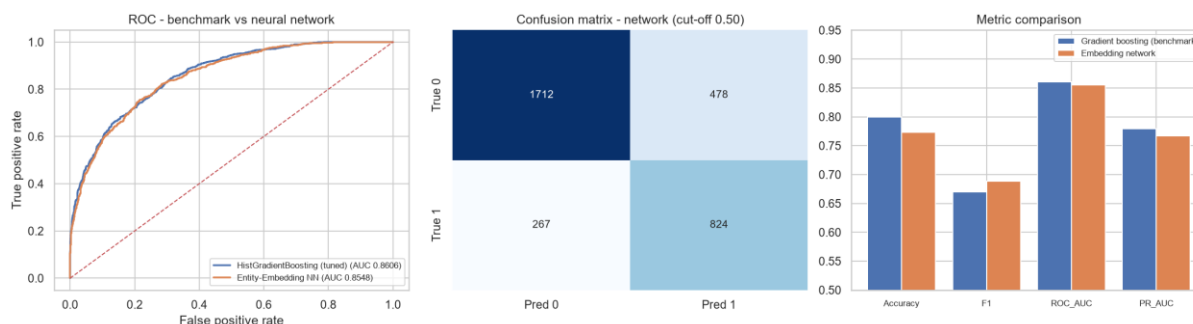


Figure 2: ROC overlay, network confusion matrix, and metric comparison.

3.1 The Headline Finding

The neural network does not beat the gradient-boosting model on ranking quality.

ROC-AUC falls from 0.8606 to 0.8548, a difference of -0.0058. Since ROC-AUC is the metric the campaign actually depends on - it scores the quality of the customer ranking - the gradient-boosting model remains the recommended production choice.

This is a well-documented result rather than a failure of implementation. On tabular data of this size and shape, gradient-boosted decision trees are a strong and difficult baseline, and neural networks typically need either substantially more data, higher-cardinality categorical fields where embeddings can compress meaningfully, or genuine sequential or interaction structure before they pull ahead. This dataset has 16,404 rows and categorical fields with at most six levels, which is close to the least favourable case for the embedding approach.

3.2 Where the Network Is Better

The comparison is not one-sided, and reporting only the AUC would hide something real. At the default 0.50 cut-off the network is markedly better on the metrics that reward finding buyers.

Metric	Gradient boosting	Embedding network	Better
Recall	0.6123	0.7553	Network
F1	0.6703	0.6887	Network
Balanced accuracy	0.7527	0.7685	Network
Precision	0.7406	0.6329	Boosting
Accuracy	0.7998	0.7729	Boosting
ROC-AUC	0.8606	0.8548	Boosting

Table 2: Metric-by-metric comparison at the default 0.50 threshold.

The reason is the `pos_weight` term in the loss, which penalises missed buyers more heavily and pushes the network to a different natural operating point - it catches 75.5 per cent of buyers against 61.2 per cent, at the cost of lower precision. That is an artefact of where the threshold sits, not evidence of a better model: threshold analysis on the benchmark showed that moving its cut-off to 0.29 raises its F1 to 0.6841, which closes most of the gap.

The honest summary is that the two models are close in overall discriminative power, and the gradient-boosting model retains a small but consistent edge on the ranking metric that matters commercially.

4. What the Embeddings Learned

This is the part of the exercise that produces something the tree ensemble could not. Each categorical level now has a learned coordinate, and those coordinates can be plotted and read.

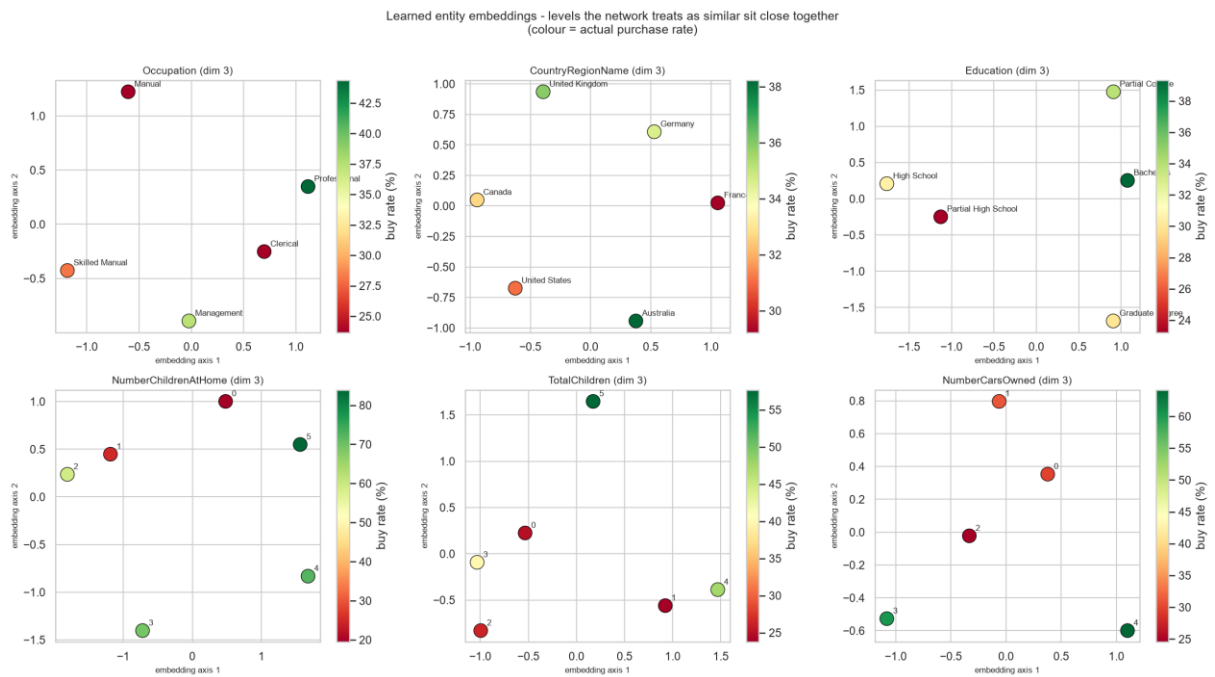


Figure 3: Learned entity embeddings, projected to two dimensions via PCA. Colour shows the actual purchase rate for each level.

4.1 The Network Organised Categories by Propensity

Nothing in the training procedure instructed the network to arrange each categorical field along a purchase-propensity axis. It did so anyway. Correlating the first embedding axis against the observed purchase rate for each level gives:

Feature	Levels	Corr_axis1_vs_buyrate	AbsCorr
Education	5	0.645	0.645
Occupation	5	0.514	0.514
NumberChildrenAtHome	6	0.388	0.388
TotalChildren	6	0.291	0.291

NumberCarsOwned	5	0.107	0.107
CountryRegionName	6	-0.062	0.062

Table 3: Correlation between embedding axis 1 and the observed purchase rate per level.

Education shows the strongest alignment at +0.645, followed by occupation. Country is the weakest, which is consistent with the earlier finding that geography carries less signal than household attributes - the network had little propensity structure to encode there, so it encoded little.

4.2 Which Levels the Network Considers Alike

Feature	ClosestPair	MostDistantPair
Occupation	Clerical <-> Professional	Professional <-> Skilled Manual
CountryRegionName	Canada <-> United Kingdom	France <-> United States
Education	Partial College <-> Bachelors	High School <-> Graduate Degree

Table 4: Closest and most distant level pairs in each embedding space.

The education result is the most striking: the closest pair is Partial College and Bachelors, and the most distant is High School and Graduate Degree. The network recovered the ordinal structure of the variable from the data alone, without being told the levels have an order.

One methodological caveat should be stated. The nearest-neighbour distances in Table 4 are computed in the full embedding space, while Figure 3 shows a two-dimensional PCA projection of it. Points that appear close in the plot are not always the closest pair in the underlying space, and the table is the authoritative one.

4.3 Marketing Implications

- Occupation and education can be collapsed into a smaller number of behavioural groups for campaign creative, using the embedding geometry rather than an analyst's intuition about which levels belong together.
- The near-zero country correlation argues against country-specific creative as a first-order lever; household composition remains the stronger axis.
- Because the embedding recovered the ordering of education unprompted, the ordinal rank encoding used in the benchmark feature set is validated as a reasonable simplification rather than an arbitrary modelling choice.

5. Conclusion and Recommendation

Recommendation: retain the gradient-boosting model in production.

- It leads on ROC-AUC (0.8606 against 0.8548), which is the metric the campaign consumes.
- It trains in seconds on CPU, with no GPU dependency and no epoch or learning-rate schedule to maintain.
- It is simpler to retrain, deploy and explain, and the accuracy difference does not justify the additional operational burden.

Retain the embedding model as an analytical instrument, not as the scorer.

- The learned embeddings are the only artefact in the project that shows which categorical levels behave alike, which is directly usable for segment design.
- The network reaches a materially higher recall operating point, which would be preferable if the business priority shifted from ranking efficiency to reaching as many buyers as possible.

Where a neural approach would be expected to win:

- Substantially more data - tens or hundreds of thousands of customers rather than sixteen thousand.
- Higher-cardinality categorical fields such as product SKU, postcode or individual store, where one-hot encoding becomes unwieldy and embeddings compress meaningfully.
StateProvinceName, dropped during feature selection for having 52 sparse levels, is exactly this kind of field and could be reintroduced as an embedding.
- Sequential behavioural data - browsing sessions, purchase histories, campaign response over time - where recurrent or attention-based architectures can exploit order that trees cannot.
- Multi-task learning - predicting purchase propensity and average monthly spend jointly from a shared representation, so the two targets regularise each other.

Appendix A - Reproducibility

Item	Value
Script	aw_deep_learning.py
Framework	PyTorch 2.7.1+cu118
Device	cuda
Random seed	42 (numpy and torch, including CUDA)
Saved model	outputs_dl/tables/dl_model.pt
Figures	outputs_dl/figures/ (3 plots)
Result tables	outputs_dl/tables/ (6 files)
Benchmark comparison source	outputs/tables/
Git repository	https://github.com/SangithaV2024BC26670/amldl-assignment-2